

CI Digital | Instituto Nacional de Telecomunicações (INATEL)

Aline, André, Ausilon, Daniella, Danilo, Erick, Guilherme, Janaína, Laís, Letícia, Lucas,
Maria, Mariana, Matheus, Nathalia, Nicolás, Rafael, Sérgio e Tarciso

Desenvolvimento e Verificação de um System-on-Chip baseado no processador PICORV32 para aplicações IoT

Relatório técnico apresentado como requisito da disciplina SD292 – Trabalho Orientado II, do Programa CI Digital, envolvendo o desenvolvimento e a verificação de um SoC com metodologia UVM.

Santa Rita do Sapucaí – MG
2026

Sumário

1 Introdução	3
2 Fundamentação Teórica	3
3 Objetivos.....	4
4 Metodologia.....	5
4.1 Organização do Projeto em Etapas	5
4.2 Ferramentas de Simulação e Desenvolvimento	5
5 Componentes	6
6 Desenvolvimento	8
6.1 AXI Interconnect	8
6.2 RAM.....	8
6.3 GPIO.....	9
6.4 UART	9
6.5 SPI.....	9
6.6 I2C	10
6.7 Temporizador	10
6.8 Bootloader	10
6.9 ROM.....	11
7 Testes	11
7.1 AXI Interconnect	12
7.2 RAM.....	12
7.3 GPIO.....	13
7.4 UART	15
7.5 SPI.....	15
7.6 I2C	16
7.7 Temporizador	17
7.8 Bootloader e ROM	19
7.8 Testes com UVM	19
8 Resultados	20
8.1 Síntese.....	22
9 Conclusão.....	24
REFERÊNCIAS BIBLIOGRÁFICAS	25

1 Introdução

A Internet das Coisas (*Internet of Things* – IoT) tem promovido uma transformação na forma como sistemas eletrônicos são projetados e utilizados, ao possibilitar a conexão de dispositivos físicos à internet para coleta, processamento e troca de dados. Aplicações em áreas como automação industrial, monitoramento de saúde e cidades inteligentes demandam sistemas embarcados capazes de operar de forma autônoma, eficiente e confiável, muitas vezes sob restrições severas de consumo de energia e recursos computacionais. Nesse contexto, a IoT vem exigindo arquiteturas de *hardware* altamente integradas e otimizadas [1].

Considerando essas exigências, torna-se necessária a adoção de plataformas computacionais capazes de integrar múltiplas funcionalidades em um único dispositivo, mantendo eficiência e flexibilidade. Nesse sentido, pode ser empregada a abordagem de *Systems-on-Chip* (SoCs). Um SoC reúne, em um único circuito integrado, elementos como unidades de processamento, memórias, periféricos e interfaces de comunicação, reduzindo a complexidade do sistema e melhorando a eficiência energética e o desempenho global. Além disso, a modularidade e a escalabilidade das arquiteturas de SoC permitem sua adaptação a diferentes cenários de aplicação, tornando essa abordagem adequada para sistemas embarcados modernos [2].

Diante desse contexto, este trabalho visa o desenvolvimento e a verificação de um SoC voltado para aplicações IoT, utilizando a linguagem Verilog e tendo como base o processador PicoRV32. O projeto busca demonstrar a aplicação prática de conceitos modernos de arquitetura de SoC e verificação funcional, alinhados às exigências atuais de sistemas embarcados conectados.

2 Fundamentação Teórica

A Internet das Coisas refere-se à interligação de dispositivos físicos inteligentes capazes de recolher, processar e trocar dados através da internet [3]. Ela constitui uma convergência multidisciplinar de tecnologias que conectam sensores, sistemas computacionais e redes. Esta abordagem possibilita a automação, monitorização remota e análise de dados em larga escala, transformando setores como indústria, saúde, agricultura e cidades inteligentes.

Contudo, a expansão massiva da IoT traz desafios importantes, incluindo segurança, eficiência, interoperabilidade e gestão de dados. A diversidade de dispositivos e protocolos exige abordagens padronizadas para garantir comunicação segura e eficiente. Neste contexto, a integração com arquiteturas SoC, processadores leves como PicoRV32, interfaces padronizadas como AXI (*Advanced eXtensible Interface*) e desenvolvimento em Verilog permite a criação de sistemas IoT eficientes, compactos e energeticamente otimizados, consolidando a convergência entre *hardware* especializado e aplicações inteligentes [4].

O PicoRV32 é um núcleo de processador RISC-V *open-source* altamente otimizado para aplicações embarcadas e FPGA (*Field-Programmable Gate Array*) [5]. Ele oferece múltiplas interfaces de comunicação e é projetado para flexibilidade e eficiência em termos de área e

desempenho. Destacando-se pela reduzida utilização de recursos lógicos e elevada frequência operacional [6].

Verilog é uma Linguagem de Descrição de *Hardware* (HDL) amplamente utilizada no projeto de sistemas digitais, descrevendo o comportamento e estrutura de circuitos eletrônicos, permitindo simulação, síntese e implementação física de *hardware* digital. A linguagem suporta modelagem comportamental, estrutural e de fluxo de dados, oferecendo flexibilidade no desenvolvimento de circuitos digitais complexos [7,8].

Um SoC integra múltiplos componentes eletrônicos — processador, memória, interfaces e periféricos — em um único chip. Esta abordagem reduz consumo energético, custo e dimensão física. O desenvolvimento de SoCs envolve ferramentas HDL, como Verilog, além de arquiteturas padronizadas de comunicação interna, como AXI. A integração eficiente destes elementos permite maior reutilização de IP cores e aceleração do desenvolvimento de hardware [9].

O AXI é um protocolo de comunicação de alto desempenho desenvolvido pela ARM como parte da especificação AMBA. Interfaces padronizadas como AXI facilitam a integração entre sensores, memória, aceleradores e processadores, simplificando o design e aumentando a interoperabilidade. Dentre as principais características do AXI destacam-se alta largura de banda, suporte a múltiplos canais independentes, transações simultâneas e baixa latência [10].

Em conclusão, a utilização de Verilog, PicoRV32, barramentos AXI e SoC representam uma abordagem moderna para sistemas embarcados e aplicações IoT eficientes e escaláveis.

3 Objetivos

Este trabalho tem como objetivo o desenvolvimento e a verificação funcional de um SoC voltado para aplicações de IoT, utilizando a linguagem Verilog e tendo como base o processador PicoRV32. O projeto busca demonstrar, de forma prática, a aplicação de conceitos modernos de arquitetura de sistemas embarcados, integrando processamento, memória e periféricos em uma única plataforma computacional, de modo a atender às exigências de eficiência, flexibilidade e confiabilidade impostas por aplicações IoT.

De forma complementar, pretende-se projetar uma arquitetura de SoC modular e escalável, composta pelo processador PicoRV32, memórias RAM (*Random Access Memory*) e ROM (*Read-Only Memory*), um módulo *bootloader* e periféricos como GPIO (*General Purpose Input/Output*), UART (*Universal Asynchronous Receiver/Transmitter*), SPI (*Serial Peripheral Interface*), I2C (*Inter-Integrated Circuit*) e temporizador, interconectados por meio do barramento AXI.

Além disso, pretende-se realizar a verificação funcional do SoC desenvolvido por meio da metodologia UVM (*Universal Verification Methodology*), assegurando o correto funcionamento tanto dos módulos individuais quanto da comunicação interna estabelecida pelo barramento.

Por fim, o projeto tem como finalidade documentar todas as etapas de desenvolvimento e verificação, descrevendo a arquitetura proposta, os métodos empregados, os testes realizados e os resultados obtidos, contribuindo para a compreensão e manutenção do sistema desenvolvido.

4 Metodologia

A metodologia adotada neste projeto é de caráter exploratório e segue uma abordagem incremental e iterativa. Ela foi definida com o objetivo de garantir o desenvolvimento e a verificação funcional de um SoC voltado para aplicações com IoT.

4.1 Organização do Projeto em Etapas

O desenvolvimento do projeto foi organizado em três etapas: *design*, verificação e documentação.

A etapa de *Design* foi responsável pelo desenvolvimento e implementação da arquitetura SoC. Nesta etapa foi definido os blocos funcionais do sistema, bem como a forma de integração entre eles, visando atender aos requisitos funcionais e arquiteturais do projeto.

A etapa de verificação foi responsável pela realização dos testes e pela garantia da validação funcional da implementação do sistema. Para a execução dos testes, foi adotada a metodologia UVM, que consiste em um padrão de verificação baseado na linguagem SystemVerilog. Essa metodologia fornece um ambiente de testes estruturado, fundamentado em boas práticas de verificação, além de permitir a reutilização e a escalabilidade dos componentes de verificação.

A etapa de documentação consistiu na elaboração de uma documentação estruturada de todo o projeto, registrando as fases de desenvolvimento e verificação, bem como os resultados obtidos e as conclusões alcançadas.

4.2 Ferramentas de Simulação e Desenvolvimento

Foram utilizadas as seguintes ferramentas para o desenvolvimento e teste deste projeto:

- Visual Studio Code: ambiente principal de edição, oferecendo suporte à escrita dos códigos em Verilog e SystemVerilog, além de auxiliar na organização e manutenção dos arquivos do projeto.
- GitHub: utilizado para o gerenciamento e versionamento do código-fonte, permitindo o acompanhamento das alterações realizadas ao longo do desenvolvimento e facilitando a organização do projeto de forma colaborativa.
- ModelSim: ferramenta utilizada para verificação funcional dos módulos individuais.
- Vivado: utilizado como ambiente de integração e suporte à metodologia UVM, possibilitando a validação do sistema em nível mais elevado, bem como a análise do comportamento do SoC durante o processo de verificação.

- OpenLane: ferramenta open-source empregada na etapa de síntese lógica e implementação física do projeto, permitindo a conversão da descrição em HDL para uma representação física do circuito.

5 Componentes

A arquitetura geral desenvolvida neste projeto é apresentada na **Figura 1**, a qual ilustra a organização estrutural do sistema e a interconexão entre seus principais módulos funcionais. No topo da arquitetura encontra-se o processador PicoRV32, responsável pela execução do *software* embarcado e pelo controle das operações do sistema, sendo sua inicialização precedida pela atuação de um módulo de *bootloader* responsável pelo carregamento do *firmware* na memória ROM. A comunicação entre o processador, a memória RAM e os periféricos é realizada por meio de um barramento AXI, implementado através de um módulo de interconexão central. A memória ROM integra a arquitetura como elemento de armazenamento inicial do *firmware*, sendo utilizada durante a fase de *boot* e não estando diretamente conectada ao barramento AXI.

Conectados ao barramento AXI estão os módulos RAM, GPIO, UART, SPI, I2C e temporizador. Cada periférico é acessado pelo processador como um dispositivo escravo, permitindo operações de leitura e escrita por meio de endereços mapeados em memória. As interfaces de comunicação (UART, SPI e I2C) possibilitam a integração do SoC com dispositivos e sensores típicos de aplicações IoT.

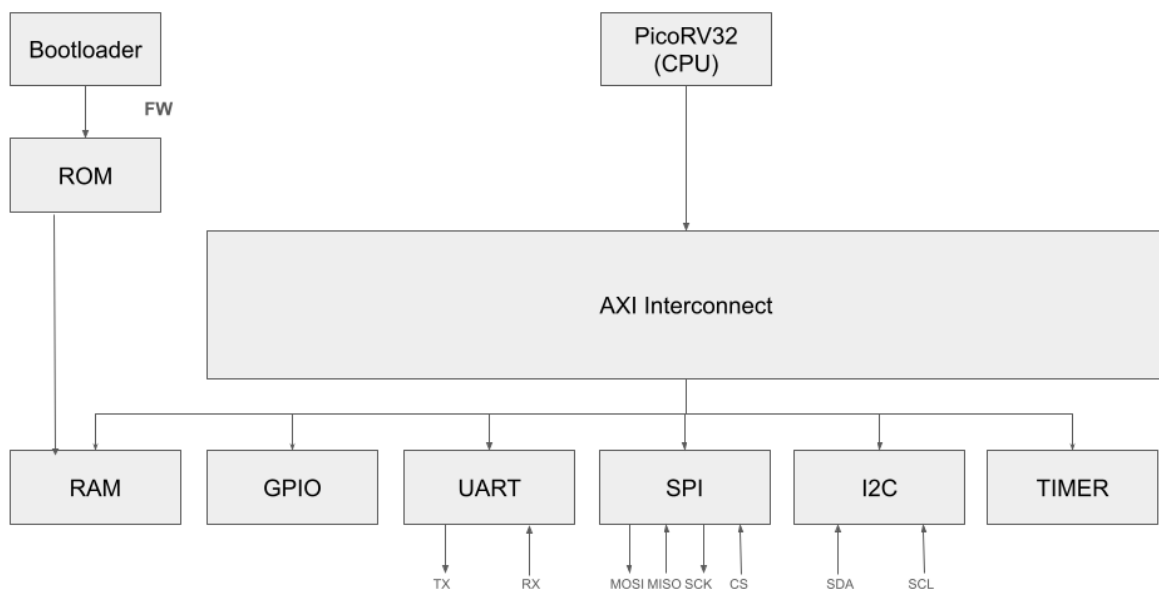


Figura 1 – Arquitetura

O mapa de endereçamento define a organização do espaço de memória acessível pelo processador, estabelecendo regiões específicas para cada periférico conectado ao barramento. A decodificação de endereços foi estruturada de forma segmentada, garantindo separação entre

memória principal e dispositivos de entrada e saída, além de permitir expansão futura da arquitetura. Também foi definida a associação das linhas de interrupção para os periféricos que geram eventos assíncronos, mantendo reservadas as interrupções internas do processador. A **Tabela 1** apresenta de forma consolidada essas informações:

Nome do Componente	Endereço Base	Tamanho da Região	Controlador de Interrupção	Nº da Porta de Interrupção
RAM	0x0000_0000	64KB	Não	-
GPIO	0x1000_0000	4KB	Sim	11
UART	0x2000_0000	4KB	Sim	12
SPI	0x3000_0000	4KB	Sim	13
I2C	0x4000_0000	4KB	Sim	14
Timer	0x5000_0000	4KB	Sim	15

Tabela 1 – Mapeamento de endereços e interrupção

A seguir, são apresentados os componentes utilizados e suas principais funções:

- PicoRV32: responsável pela execução do *software* embarcado, controle dos periféricos e coordenação das operações do sistema.
- Barramento AXI: a comunicação interna entre processador, RAM e periféricos é realizada por meio do protocolo AXI-Lite, uma versão simplificada do padrão AXI, adequada para acesso a registradores e dispositivos de controle. Este módulo é responsável por realizar o roteamento das transações entre o mestre (PicoRV32) e os diversos dispositivos escravos, garantindo o correto endereçamento e sincronização das operações de leitura e escrita.
- Memória RAM: utilizada para armazenamento de dados e instruções durante a execução do *software* embarcado. Atua como principal área de armazenamento volátil do sistema, sendo empregada para a alocação de variáveis, dados temporários e estruturas utilizadas pelas aplicações executadas no processador. Sua presença é essencial para o correto funcionamento do *software* embarcado, viabilizando a execução de rotinas de controle, comunicação e processamento de dados.
- GPIO: fornece ao sistema a capacidade de leitura e escrita de sinais digitais externos. Esse periférico é utilizado para interação básica com o ambiente, como controle de sinais de saída e leitura de estados externos.
- UART: responsável pela comunicação serial assíncrona do SoC com dispositivos externos. O módulo implementa tanto a transmissão quanto a recepção de dados, possibilitando a troca de informações para depuração, monitoramento ou comunicação com outros sistemas.
- SPI: periférico de comunicação serial que permite a comunicação *full-duplex* síncrona com dispositivos periféricos externos, como sensores e memórias.
- I2C: interface utilizada em sistemas embarcados para comunicação com sensores e dispositivos de baixa velocidade, atuando como escravo no barramento, permitindo que o processador configure e leia dados de dispositivos externos de forma mais eficiente.
- Temporizador: fornece mecanismos de contagem e geração de eventos periódicos, sendo fundamental para o controle de tempo, agendamento de tarefas e geração de interrupções.

- *Bootloader*: responsável pelo gerenciamento da fase de inicialização do sistema. O módulo recebe o *firmware* via interface UART, realiza a reconstrução dos dados transmitidos serialmente e controla o processo de gravação na memória ROM.
- Memória ROM: utilizada como elemento de armazenamento inicial do firmware durante a fase de *boot*. Posteriormente, os dados armazenados são disponibilizados ao sistema para execução do programa pelo processador, integrando o fluxo de inicialização da arquitetura.

6 Desenvolvimento

O desenvolvimento do SoC foi conduzido de forma modular, priorizando organização estrutural, reutilização de componentes e escalabilidade da arquitetura. A implementação de cada componente está descrita nos tópicos a seguir:

6.1 AXI Interconnect

O funcionamento do *AXI Interconnect* baseia-se na decodificação do endereço presente nas transações geradas pelo processador. A partir desse endereço, o módulo identifica qual dispositivo escravo deve responder à requisição e direciona os sinais de controle e dados correspondentes. Dessa forma, garante-se que apenas o módulo selecionado receba os sinais de habilitação e execute a operação solicitada, enquanto os demais permanecem inativos.

Durante uma operação de escrita, o *Interconnect* encaminha os sinais de endereço, dados e controle ao dispositivo escravo correspondente, aguardando a resposta de confirmação da transação. De maneira semelhante, em operações de leitura, o módulo direciona a requisição ao escravo apropriado e retorna ao processador os dados recebidos, assegurando a integridade e sincronização da comunicação.

6.2 RAM

O módulo de memória RAM foi construído de modo parametrizável quanto à largura de endereço e à largura de dados, sendo configurado no projeto com palavras de 32 bits. O acesso à RAM é realizado por meio dos cinco canais definidos pelo protocolo AXI4-Lite: canal de endereço de escrita, canal de dados de escrita, canal de resposta de escrita, canal de endereço de leitura e canal de dados de leitura. A implementação segue o mecanismo de *handshake*, garantindo a sincronização entre mestre e escravo durante as transações.

Durante operações de escrita, o módulo aguarda a validação simultânea do endereço e dos dados para habilitar o armazenamento da informação na posição correspondente da memória. A escrita é realizada de forma seletiva por *byte*, o que permite modificar apenas os *bytes* desejados dentro da palavra de 32 bits. Após a conclusão da operação, o módulo gera uma resposta indicando sucesso da transação.

Nas operações de leitura, o endereço solicitado é decodificado e o conteúdo correspondente é disponibilizado no canal de dados de leitura. O endereçamento interno considera alinhamento por palavra, descartando os dois *bits* menos significativos do endereço para acessar corretamente a posição da memória.

6.3 GPIO

O periférico GPIO foi implementado como registrador mapeado em memória. Operações de escrita atualizam o registrador de saída e refletem imediatamente no sinal físico, enquanto operações de leitura retornam o último valor armazenado. Além disso, o módulo também suporta escrita parcial por *byte*.

Este módulo também permite que o *firmware* embarcado execute um teste mínimo de comunicação. Padrões previamente definidos de dados são escritos no registrador do GPIO e, em seguida, lidos para verificação e comparação com os valores esperados. Em caso de inconsistência, o programa permanece em laço infinito indicando falha.

6.4 UART

O módulo foi desenvolvido com o objetivo de integrar uma interface de comunicação serial UART a um sistema baseado em barramento AXI-Lite, permitindo que um processador ou mestre AXI controle a transmissão e recepção de dados por meio de registradores mapeados em memória.

A arquitetura separa a lógica do barramento AXI da comunicação serial, utilizando registradores internos para armazenar o dado a ser transmitido, controlar o início da transmissão e indicar o estado de ocupação do periférico. A escrita no endereço 0x000 inicia automaticamente o envio de um *byte* pela UART, sendo aceita apenas quando o transmissor está livre.

A comunicação serial é realizada por módulos independentes de transmissão e recepção, responsáveis por converter dados paralelos em seriais e vice-versa. Para leitura, o sistema disponibiliza registradores que retornam o último dado recebido (0x004) e informações de status (0x008), indicando recepção concluída e disponibilidade do transmissor.

6.5 SPI

O funcionamento do módulo se dá por um conjunto de registradores internos acessíveis via AXI, que permitem o controle da transmissão e a recepção dos dados. O processador pode habilitar o funcionamento da interface por meio de um registrador de controle, escrever o dado a ser transmitido no registrador de transmissão e verificar o estado da comunicação por meio de sinais de status. Além disso, o dado recebido durante a comunicação é armazenado em um registrador de recepção, podendo ser posteriormente lido pelo software.

Quando o processador escreve um valor no registrador e habilita a interface, o módulo SPI inicia automaticamente a comunicação, assumindo o papel de mestre. Nesse momento, o sinal *spi_cs* é colocado em nível ativo, indicando ao dispositivo escravo o início da transação. Em seguida, o módulo passa a gerar o sinal de *clock* serial, responsável por sincronizar a troca de dados entre mestre e escravo. A implementação permite que o envio e o recebimento de dados ocorram ao mesmo tempo.

O módulo utiliza registradores de deslocamento para enviar e receber os bits sequencialmente. A cada ciclo de *clock* serial, um bit é transmitido ao dispositivo externo e, simultaneamente,

um bit é recebido. Um contador interno controla o número de bits transmitidos, garantindo que a operação seja concluída após oito ciclos, correspondentes a um *byte* completo. Ao final da transmissão, o módulo desativa o sinal de seleção do escravo, armazena o dado recebido e sinaliza que a interface está disponível para uma nova operação.

6.6 I2C

O módulo I2C possui três interfaces: *clock* e *reset*; interface AXI *Slave* que é responsável pela comunicação com o processador tendo canais de escrita e leitura; e a interface física que através de SDA (*Serial Data Line*) e SCL (*Serial Clock Line*) comunica com dispositivos externos.

O módulo possui também dois registradores acessíveis via AXI para armazenar o endereço do dispositivo I2C de destino e para armazenar o dado a ser transmitido. Além disso, opera com uma máquina de estado finitos (FSM) implementada para controlar todo o processo de comunicação. Os estados implementados são:

- **IDLE** – Estado inicial, aguardando comando do processador.
- **START** – Geração da condição de início do protocolo (SDA transita para LOW enquanto SCL está HIGH).
- **SEND_ADDR** – Envio sequencial dos 8 bits do endereço armazenado em *addr_reg*.
- **ACK_ADDR** – Ciclo de reconhecimento (ACK) do escravo após o envio do endereço.
- **SEND_DATA** – Envio dos 8 bits do dado armazenado em *data_reg*.
- **ACK_DATA** – Novo ciclo de reconhecimento do escravo após o envio do dado.
- **STOP** – Geração da condição de parada (SDA retorna para HIGH enquanto SCL está HIGH).

Também foram implementados um divisor de *clock*, para adequar a velocidade do barramento I2C, e o comando *busy* para impedir o processador de enviar comandos enquanto uma transação I2C estiver em andamento.

6.7 Temporizador

Este módulo é um temporizador programável, responsável por gerar a contagem de ciclos de *clock* e gerar interrupções periódicas para o processador. O funcionamento dele se dá por um contador interno que é incrementado a cada ciclo de *clock*. O valor máximo da contagem é definido por um registrador configurável e quando o contador atinge esse valor limite, o temporizador reinicia automaticamente a contagem e sinaliza a ocorrência do evento por meio de uma interrupção. Essa interrupção é enviada ao processador através do sinal *irq_out*, indicando que uma ação previamente programada deve ser executada.

6.8 Bootloader

O *bootloader* implementado é composto por um módulo externo de transmissão e um módulo interno de gerenciamento. O módulo *bootloader_uart*, realiza a leitura do arquivo .hex e transmite seu conteúdo sequencialmente via UART para o SoC, simulando um processo de gravação de memória. O módulo *boot_manager* recebe os dados seriais, reconstrói as palavras e realiza a gravação na memória ROM. Durante essa etapa, o sistema opera em modo de

gravação, controlado pelo sinal *boot_mode*. Ao término da transmissão, o sinal *done* indica a finalização do processo, permitindo a transição para o modo de execução.

6.9 ROM

A memória ROM foi implementada como um módulo dedicado de armazenamento, parametrizável quanto à largura de dados e profundidade, permitindo adaptação ao tamanho do *firmware* transmitido. Internamente, foi descrita como um arranjo de registradores indexados por endereço, com escrita síncrona controlada pelo módulo *boot_manager*. Durante o modo de gravação, o *boot_manager* realiza o armazenamento sequencial das palavras recebidas. O endereço interno é incrementado automaticamente a cada dado válido recebido, garantindo a organização linear do *firmware* na memória.

7 Testes

A estratégia adotada para os testes contemplou simulações funcionais em nível de módulo e testes integrados com execução de *firmware* na memória RAM.

Inicialmente, cada periférico foi validado isoladamente por meio de simulações executadas no ModelSim, observando-se os sinais de controle, temporização e resposta às transações AXI. Foram verificados os mecanismos de leitura e escrita, a correta decodificação de endereços no AXI e o funcionamento do protocolo de *handshake*. E, para os módulos com geração de interrupção, foi analisada a ativação correta do sinal de *irq* diante dos eventos programados.

Posteriormente, a verificação foi conduzida em nível de sistema, priorizando a interação entre CPU, barramento e periféricos. Foram definidos os seguintes casos de testes básicos para cada protocolo de entrada: despertar por interrupção de temporizador, leitura de dados via periféricos, processamento, transmissão das informações e retorno ao modo de baixo consumo. Para estruturar essa verificação foi adotada a metodologia UVM, promovendo separação entre geração de estímulos, monitoramento e verificação funcional, com execução no *software* Vivado.

No nível superior, o *Testbench Top* instancia o DUT (*Device Under Test*) e conecta as interfaces. Essa interação com o DUT ocorre por meio de uma BFM (*Bus Functional Model*), implementada como interface. O ambiente principal agrega os *agents*, o *scoreboard* e os *coverages*. Cada *agent* é composto por *sequencer*, *driver* e *monitor*, podendo operar no modo ACTIVE, com *driver*, ou PASSIVE, apenas com monitor. O *sequencer* coordena a execução das *sequences*, o *driver* converte transações em sinais aplicados ao DUT, e o monitor observa passivamente e coleta transações. Complementarmente, foram implementados mecanismos de cobertura funcional e de código.

Além disso, há o *scoreboard* que realiza a comparação entre os resultados esperados e os valores efetivamente observados, utilizando um modelo de referência para detectar inconsistências funcionais.

7.1 AXI Interconnect

A validação do AXI *Interconnect* foi realizada por meio de um *testbench* comportamental contendo modelos simplificados dos *slaves*. Ela realiza geração de *clock*, *reset* síncrono, estímulos controlados para leitura e escrita e verificação automática dos sinais de controle e dados. Foram realizados os testes de escrita e leitura na RAM, leitura e escrita de GPIO, contemplando também a escrita e leitura em endereço inválido. A **Figura 2** demonstra o resultado do teste realizado:

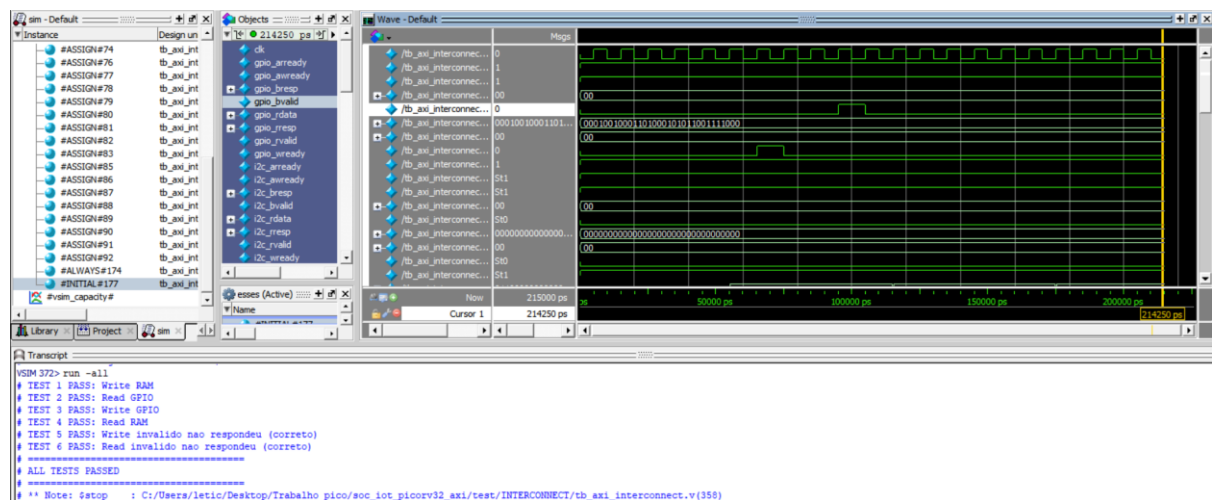


Figura 2 – Teste do AXI Interconnect

7.2 RAM

Para o módulo RAM, foram propostos dois casos de teste:

- Teste básico sem CPU (Figura 3):** *testbench* que simula um mestre AXI genérico interagindo diretamente com a RAM. Não inclui o processador PicoRV32. Os cenários testados incluem escrita em endereços específicos, leitura dos mesmos endereços para verificar integridade, teste de *strobe* para escritas parciais e teste de *reset* e inicialização.

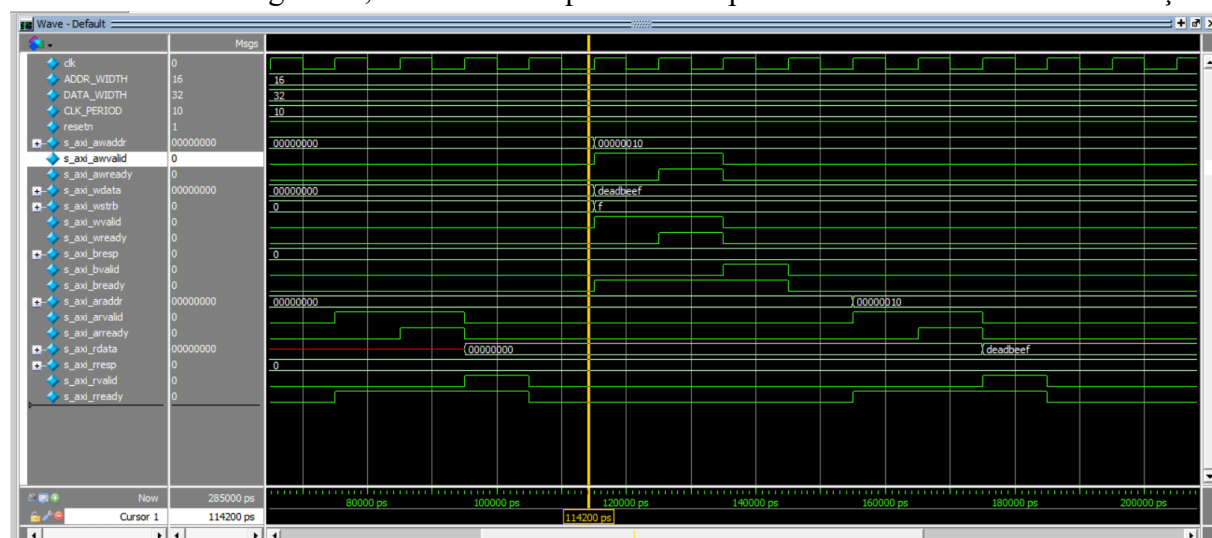


Figura 3 – Teste da RAM sem integração com CPU

- b. **Teste com CPU PicoRV32 (Figura 4):** *testbench* que conecta o processador PicoRV32 à RAM via interconexão AXI. Os cenários testados incluem inicialização da RAM com o arquivo *firmware.hex*, execução de instruções RISC-V, validação de transações AXI geradas pelo CPU e verificação de comportamento em *loop*.

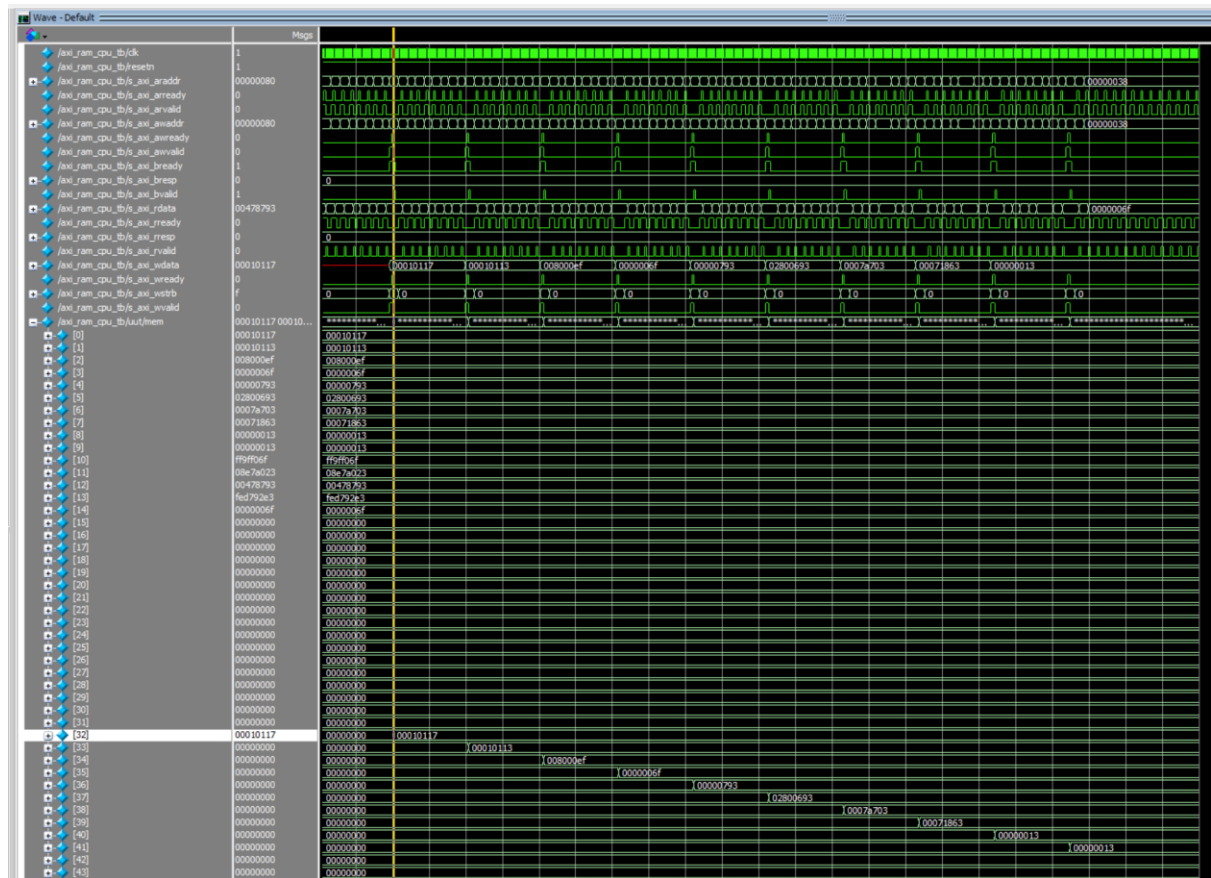


Figura 4 – Teste da RAM com integração com CPU

7.3 GPIO

O módulo GPIO foi testado por meio de uma simulação em nível de sistema, utilizando uma *testbench* responsável por gerar *clock* e *reset*, permitindo a inicialização completa do SoC e do periférico. As operações foram realizadas por meio de transações AXI-Lite geradas tanto pelo PicoRV32 quanto por tarefas de monitoramento da *testbench*, reproduzindo o acesso a registradores mapeados em memória e validando a comunicação entre CPU, AXI e GPIO.

Para validar o mapeamento e a decodificação de endereços, foram executadas escritas e leituras no endereço base do GPIO, monitorando os canais do barramento AXI. A validação funcional do periférico foi realizada por meio de escritas no registrador interno do GPIO e posterior leitura de retorno. Foram utilizados padrões conhecidos de dados, além de testes com escrita parcial utilizando *Write Strobes* para *byte* e *halfword*. Após cada escrita, o valor de saída foi monitorado para garantir que o registrador mantinha o valor *latched* e que as leituras retornavam exatamente o conteúdo armazenado.

Para validar a integração com a CPU, foi executado *firmware bare-metal* no PicoRV32 realizando operações de *write* e *readback* via acesso MMIO (*Memory-mapped I/O*). O *firmware* escreve valores no registrador do GPIO e lê de volta o conteúdo, entrando em *loop* infinito em caso de inconsistência.

A *testbench* também implementou um mecanismo de verificação automática baseado em monitoramento do barramento AXI, observação direta do *gpio_out*, validação de sequência de escrita esperada e *timeout* de segurança. A **Figura 5 e 6** trazem o resultado do teste realizado:

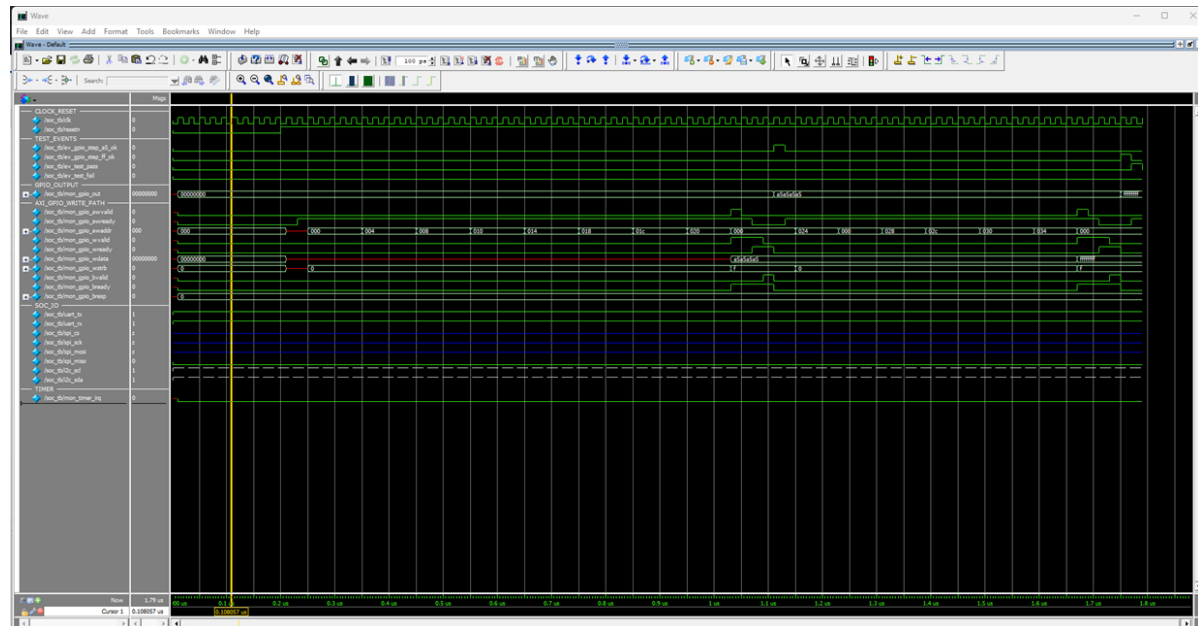


Figura 5 – Teste do GPIO: saída gráfica da simulação

```

VSIM 18> run -all
[2000000] [TEST_GPIO] [INFO] Inicio: aguardando GPIO_OUT ASASASAS -> FFFFFFFF
===== MONITOR START =====
=== Simulacao Iniciada ===
[1050000] [AXI][GPIO] AW addr=0x000
[1090000] [AXI][GPIO] W data=0xa5a5a5a5 wstrb=0xf
[1110000] [AXI][GPIO] B resp=0x0 (OK=00)
[1110000] [TEST_GPIO] [STEP1] [OK] GPIO_OUT atingiu 0xASASASAS
[1130000] [GPIO_MON] gpio_out = 0xa5a5a5a5
[1130000] [EVENT] STEP1_OK (ASASASAS)
[1690000] [AXI][GPIO] AW addr=0x000
[1730000] [AXI][GPIO] W data=0xffffffff wstrb=0xf
[1750000] [AXI][GPIO] B resp=0x0 (OK=00)
[1750000] [TEST_GPIO] [STEP2] [OK] GPIO_OUT atingiu 0xFFFFFFFF
[1770000] [GPIO_MON] gpio_out = 0xffffffff
[1770000] [EVENT] STEP2_OK (FFFFFFFF)
[1790000] [EVENT] >>> TEST PASS <<<
[1790000] [TEST_GPIO] [RESULT] [OK] PASS

===== TEST SUMMARY =====
TEST_GPIO : PASS
-----
TOTAL=1 PASS=1 FAIL=0
=====

[TB] RESULT: ALL TESTS PASSED.
** Note: $finish : C:/Projetos/SoC_TESTES/tb_soc_top.sv(409)
Time: 1790 ns Iteration: 1 Instance: /soc_tb

```

Figura 6 – Teste do GPIO: saída textual da simulação

7.4 UART

Para a validação do módulo UART foram realizados dois testes utilizando *testbenches*. O primeiro teste foi realizado para validação funcional inicial do sistema, com foco em confirmar o correto funcionamento da transmissão serial. Nessa abordagem, os dados são enviados ao periférico por meio de escritas AXI e o sinal físico de saída (tx) é monitorado para reconstrução do quadro UART transmitido. Os bytes capturados são comparados com valores previamente definidos, permitindo identificar erros de enquadramento ou divergências de dados.

A segunda *testbench* foi desenvolvida em SystemVerilog com uma abordagem de verificação mais completa e estruturada, ampliando o escopo dos testes para além da funcionalidade básica. Nesse ambiente, foram incluídas verificações automáticas do protocolo AXI por meio de *assertions*, além da separação modular entre *drivers* UART, mestres AXI e sequências de teste. Foram avaliadas diferentes condições de operação, incluindo transmissão, recepção, injeção de erro de *framing* e testes de estresse com variação controlada do tempo de bit, permitindo medir a robustez temporal do receptor. Os resultados deste teste estão apresentados na **Figura 7**.

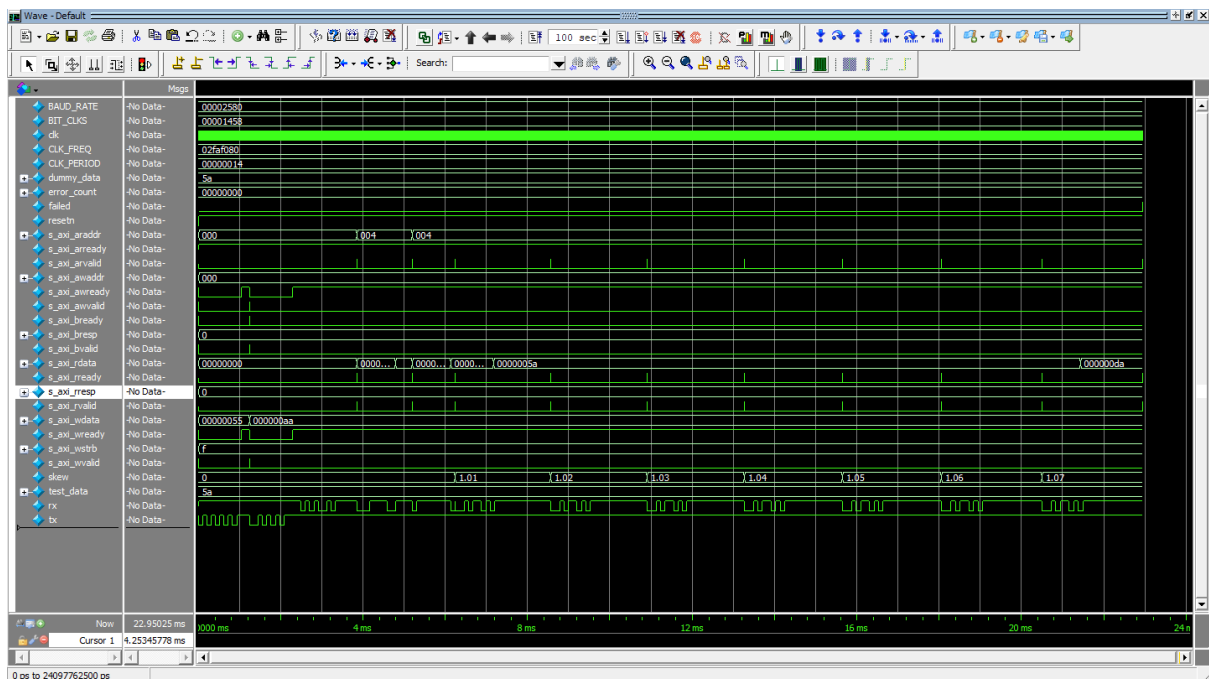


Figura 7 – Teste da UART

7.5 SPI

O módulo SPI foi testado por meio de uma simulação utilizando uma *testbench* que gera *clock* e *reset*, permitindo a inicialização correta do periférico. As operações foram realizadas através de transações AXI-Lite simuladas, implementadas por tarefas de leitura e escrita que reproduzem o acesso de um processador aos registradores do módulo. Cada transferência é iniciada pela escrita do dado no registrador de transmissão e pela habilitação do SPI, enquanto a *testbench* monitora o registrador de status até que o sinal de ocupado indique o término da comunicação.

Para validar a comunicação serial, foi implementado um escravo SPI simulado operando no modo 0, responsável por receber os dados enviados em MOSI (Master Out, Slave In) e retornar respostas controladas via MISO (Master In, Slave Out). Diversos testes sequenciais com diferentes padrões de dados foram executados, permitindo verificar a geração correta do *clock* SPI, o acionamento do sinal chip *select*, o deslocamento dos bits e o armazenamento dos dados recebidos. O resultado obtido pode ser observado na **Figura 8**.

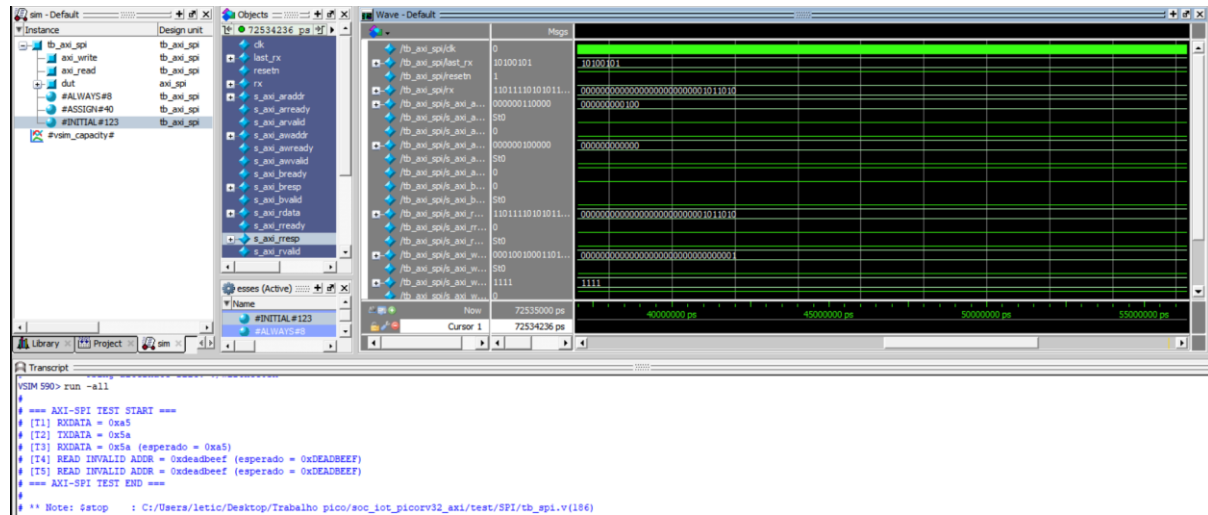


Figura 8 – Teste do SPI

7.6 I2C

A validação do módulo I2C foi realizada por meio de simulação utilizando uma *testbench* dedicada ao periférico AXI-I2C. Essa *testbench* gera *clock* e *reset*, além de implementar tarefas de leitura e escrita AXI-Lite que reproduzem o acesso de um mestre aos registradores mapeados em memória. Foram executados testes de configuração do *prescaler* e registrador de controle, escrita do endereço do escravo, envio de dados pelo registrador de transmissão e leitura do registrador de status, verificando o correto *handshake* dos canais AXI e a integridade das transações.

Em seguida, foi executada uma sequência completa de comunicação I2C contendo habilitação do módulo, geração de condição START, envio de endereço com bit de escrita, transmissão de dados e geração de condição STOP. A simulação permitiu observar a correta geração do sinal SCL, o comportamento *open-drain* da linha SDA e a propagação serial dos bits conforme o protocolo I2C. Adicionalmente, o módulo foi validado em nível de sistema por meio da integração ao SoC, utilizando um modelo comportamental de escravo I2C implementado na *testbench*. Esse modelo detecta automaticamente as condições de START e STOP, captura os *bytes* transmitidos e responde com sinal de ACK, permitindo confirmar a correta comunicação entre mestre e escravo. Os resultados da simulação estão apresentados na **Figura 9 e 10**.

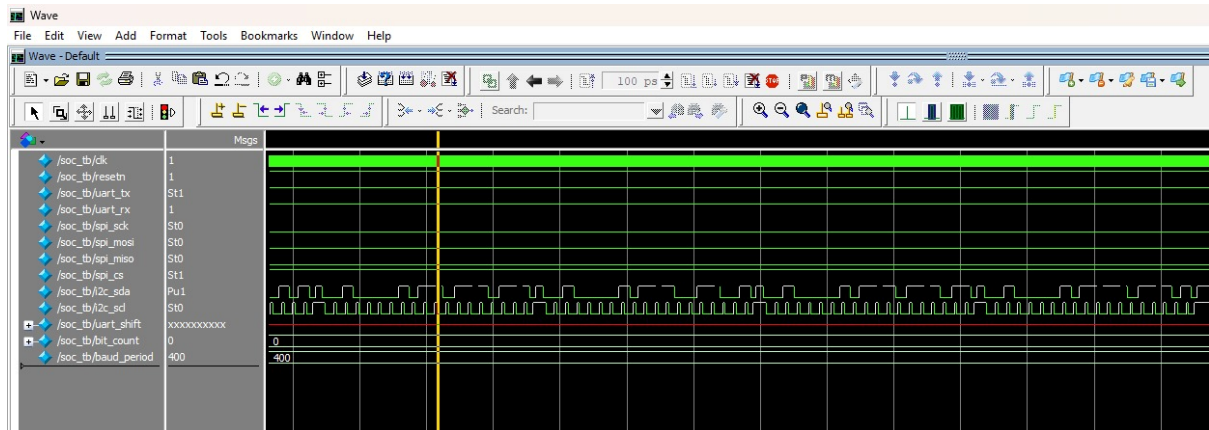


Figura 9 – Teste 1 da I2C

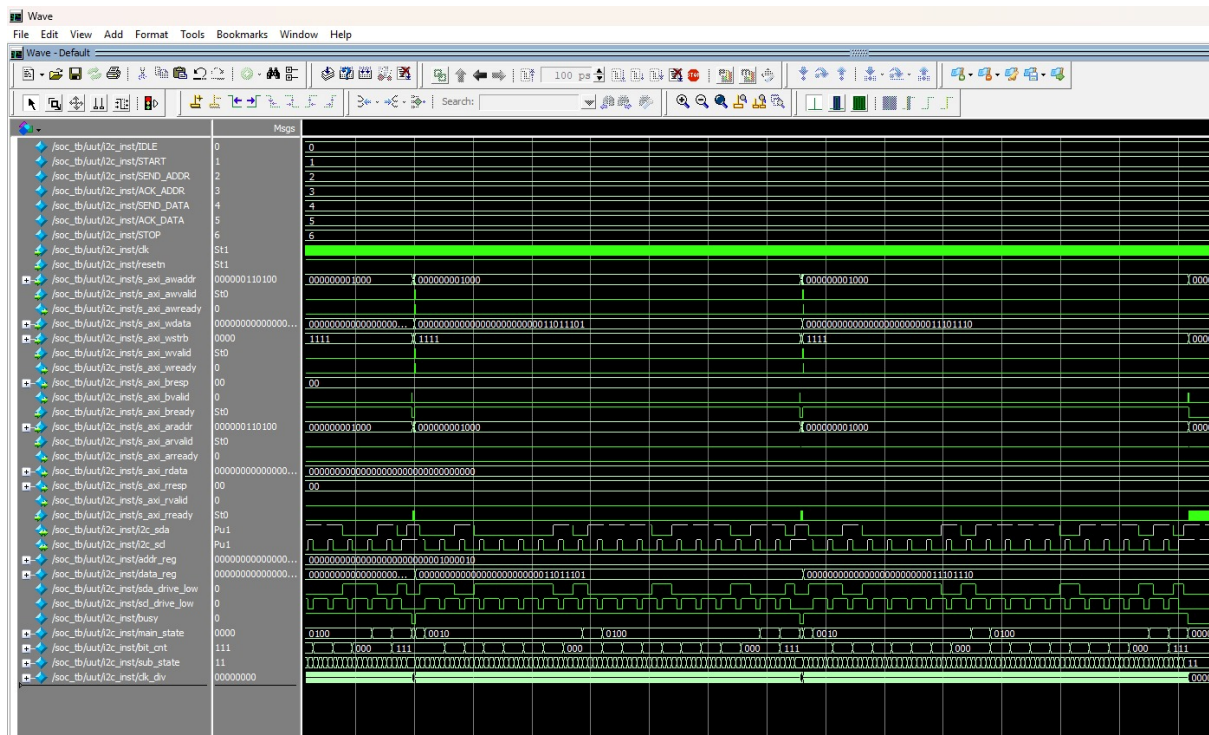


Figura 10 – Teste 2 da I2C

7.7 Temporizador

O módulo temporizador foi validado por meio de uma simulação em nível de sistema, sendo integrado ao SoC e testado através de uma *testbench* que gera *clock*, aplica *reset* e monitora a saída UART. Após a inicialização, o processador configura o temporizador via interface AXI-Lite, e o correto funcionamento é verificado indiretamente pela recepção sequencial dos caracteres “A”, “B” e “C”, enviados pelo *software* somente após a ocorrência das interrupções geradas pelo *timer*. Cada caractere representa uma etapa distinta de operação, incluindo temporização inicial, funcionamento periódico e reconfiguração dinâmica do valor de recarga. Um mecanismo de verificação e *timeout* garante a detecção de falhas, permitindo validar tanto a lógica interna do temporizador quanto sua integração com o barramento AXI e o tratamento de interrupções pelo sistema.



Figure 12. Teste de temporização código textual de simulação

7.8 Bootloader e ROM

O funcionamento do *bootloader* e da ROM foi verificado por meio da correta execução do *firmware* escrito. Uma vez finalizada a etapa de *boot* e iniciada a execução, o comportamento esperado do sistema — incluindo acesso à memória, acionamento de periféricos e geração de sinais conforme o programa carregado — confirma que a recepção, armazenamento e disponibilização do *firmware* ocorreram de forma íntegra e consistente.

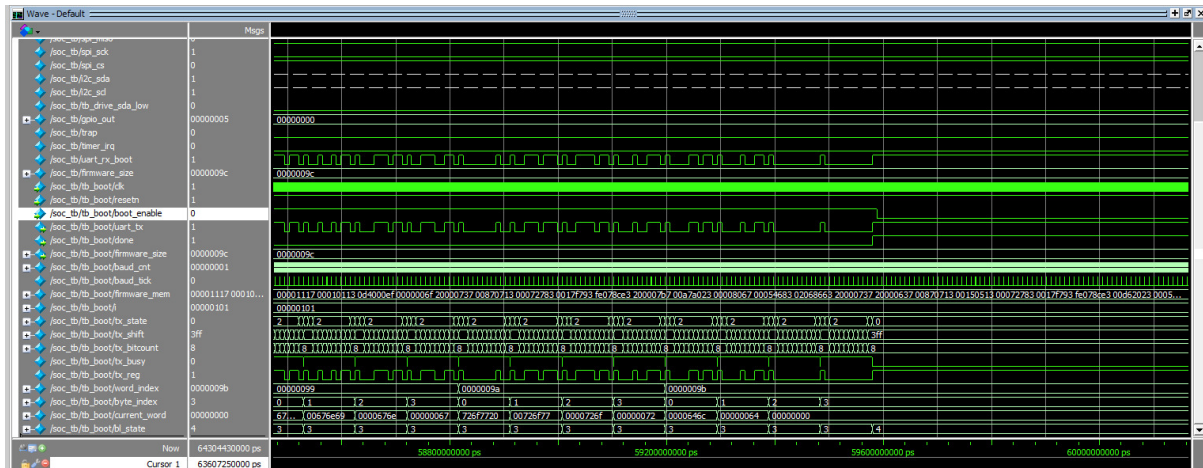


Figura 13 – Teste do bootloader e da ROM

7.8 Testes com UVM

Para garantir uma verificação estruturada e reutilizável do sistema, foi desenvolvido um ambiente baseado na metodologia UVM. O ambiente é composto por agentes dedicados ao *bootloader* e aos periféricos, cada um estruturado com *sequencer*, *driver*, monitor e *transaction*, além de mecanismos de cobertura funcional e um *scoreboard* central responsável pela checagem dos resultados. A comunicação com o DUT ocorre por meio de um BFM, permitindo a aplicação de estímulos controlados e a análise sistemática do comportamento do sistema sob teste, conforme ilustrado na **Figura 14**.

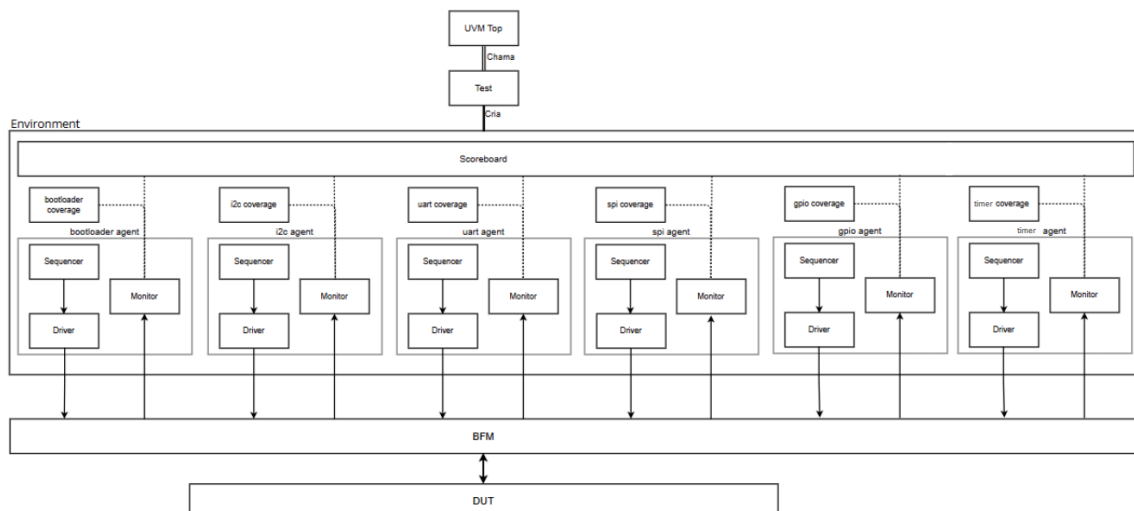


Figura 14 – Estrutura UVM

A verificação das interfaces foi conduzida por meio do fluxo padrão de estímulo da metodologia UVM, no qual o *test* instancia e executa sequências responsáveis por gerar transações em uma ordem previamente definida. Essas transações são encaminhadas ao *sequencer*, que as organiza em fila e as envia individualmente ao *driver*. O *driver*, por sua vez, converte cada transação em sinais elétricos por meio do BFM, aplicando-os ao DUT para validação funcional, conforme o diagrama presente na **Figura 15**.

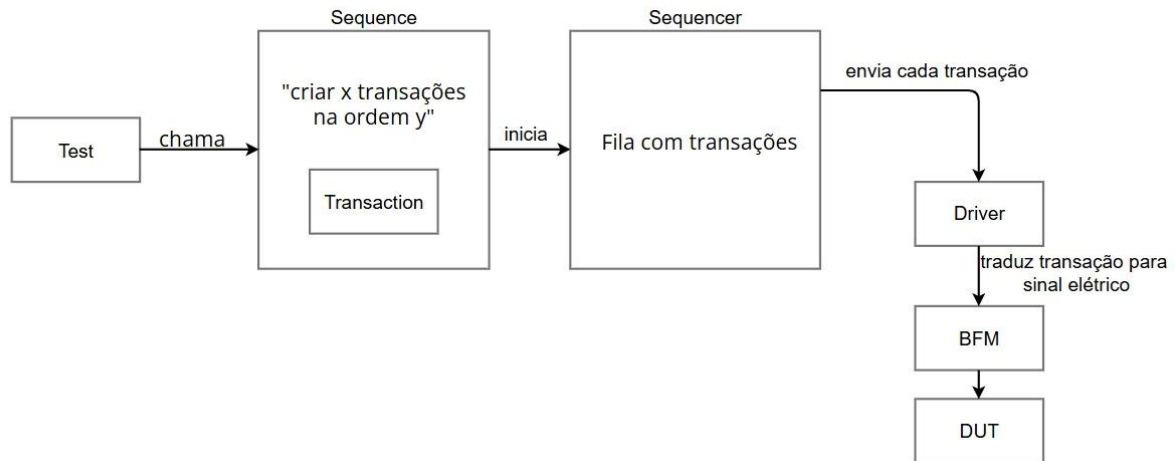


Figura 15 – Diagrama de funcionamento da UVM

Para análise do funcionamento geral do SoC, primeiramente o *bootloader* foi acionado e, assim que seu funcionamento foi validado pelo sinal de *done*, estímulos foram enviados via UART e as respostas recebidas pelos periféricos. O comportamento esperado está descrito abaixo:

- Mensagem inicial → o SoC envia “SOC IOT PICORV32” na saída UART;
- Recebendo ‘A’ → o SoC envia ‘A’ na saída GPIO;
- Recebendo ‘B’ → o SoC envia ‘5’ na saída GPIO e a sequência “good morning world” na saída SPI;
- Recebendo ‘C’ → o SoC envia ‘8’ na saída GPIO e os valores 11, 22 33 na saída I2C;
- Recebendo ‘D’ → o SoC envia ‘F’ na saída GPIO e ‘DD\n’ na saída UART;
- Recebendo ‘E’ → o SoC envia ‘1’ na saída GPIO e configura um timer de 5000, em seguida, envia ‘E’ na saída GPIO.

Esse teste valida o roteamento de dados entre protocolos e a correta interpretação de comandos pelo firmware do SoC.

8 Resultados

Esta seção apresenta os resultados obtidos ao longo do desenvolvimento e verificação do SoC proposto, contemplando tanto os testes funcionais realizados em simulação quanto os dados provenientes da síntese do projeto na ferramenta OpenLane.

A **Tabela 2** consolida os resultados obtidos para cada módulo, considerando testes individuais, testes integrados e validação via ambiente UVM.

Módulo	Teste Individual	Teste Integrado	Teste UVM	Resultado Geral
RAM	Aprovado	Aprovado	Aprovado	Funcional
AXI Interconnect	Aprovado	Aprovado	Aprovado	Funcional
GPIO	Aprovado	Aprovado	Aprovado	Funcional
UART	Aprovado	Aprovado	Aprovado	Funcional
SPI	Aprovado	Aprovado	Aprovado	Funcional
I2C	Aprovado	Aprovado	Aprovado	Funcional
Temporizador	Aprovado	Aprovado	Aprovado	Funcional
ROM e Bootloader	Aprovado	Aprovado	Aprovado	Funcional

Tabela 2 – Resultado dos testes

A **Figura 16** exibe o relatório final gerado pelo *scoreboard* do ambiente de verificação UVM após a execução completa dos testes. A partir das informações reportadas, observa-se que o sistema executou corretamente as transações previstas para os periféricos integrados ao SoC. As formas de onda correspondentes às transações realizadas durante a simulação são apresentadas na **Figura 17**, permitindo observar detalhadamente o comportamento temporal dos sinais e a interação entre os módulos do sistema.

```
[SCOREBOARD] Recebeu da GPIO: 0x00000008 depois de 0x43
[SCOREBOARD] I2C received: #0: Addr=0x20 Data=0x11
[SCOREBOARD] I2C received: #1: Addr=0x20 Data=0x22
[SCOREBOARD] I2C received: #2: Addr=0x20 Data=0x33
[SCOREBOARD] UART Transaction sent to DUT #4: Data=0x44
[SCOREBOARD] Recebeu da GPIO: 0x0000000f depois de 0x44
[SCOREBOARD] UART Transaction sent to DUT #5: Data=0x45
[SCOREBOARD] Recebeu da GPIO: 0x00000001 depois de 0x45
[SCOREBOARD] Recebeu da GPIO: 0x0000000e depois de 0x00
[VIRTUAL SEQUENCE] Tests finished

[SCOREBOARD] ===== Scoreboard Report =====
[SCOREBOARD] Bootloader: 4239321 cycles
[SCOREBOARD] Initial message received: SOC IOT PICORV32
[SCOREBOARD] ----- UART -----
[SCOREBOARD] UART write on DUT: 5 transactions
[SCOREBOARD] UART read from DUT: 1 transactions
[SCOREBOARD] UART message correctly received: 1
[SCOREBOARD] UART message wrongly received: 0
[SCOREBOARD] ----- GPIO -----
[SCOREBOARD] GPIO read from DUT: 6 transactions
[SCOREBOARD] GPIO message correctly received: 6
[SCOREBOARD] GPIO message wrongly received: 0
[SCOREBOARD] ----- SPI -----
[SCOREBOARD] SPI write on DUT: 0 transactions
[SCOREBOARD] SPI read from DUT: 1 transactions
[SCOREBOARD] SPI message correctly received: 1
[SCOREBOARD] SPI message wrongly received: 0
[SCOREBOARD] ----- I2C -----
[SCOREBOARD] I2C: 3 transactions
[SCOREBOARD] I2C message correctly received: 3
[SCOREBOARD] I2C message wrongly received: 0
[SCOREBOARD] ----- TIMER -----
[SCOREBOARD] Timer: 1 transactions
[SCOREBOARD] Delay in clock cycles: 5048
[SCOREBOARD] Delay difference: 48
[SCOREBOARD] Delay error: 0
[SCOREBOARD] -----
[SCOREBOARD] Errors: 0
[SCOREBOARD] =====
[SCOREBOARD] Test completed with NO errors!
```

Figura 16 – Scoreboard da UVM

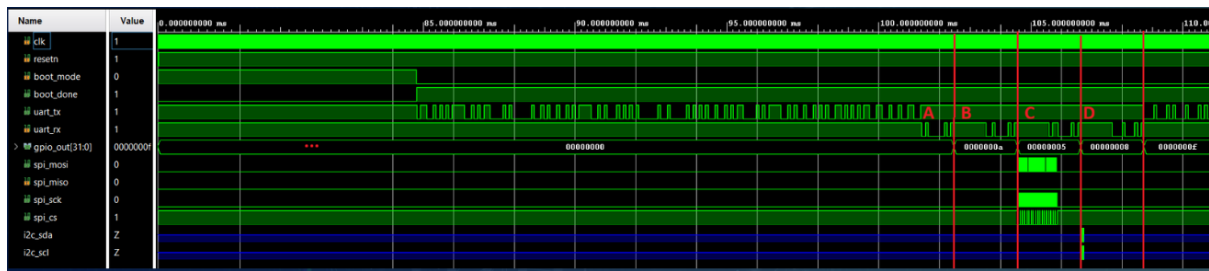


Figura 17 – Simulação do SoC

Os resultados demonstram que os módulos atenderam aos requisitos funcionais estabelecidos, apresentando comportamento consistente tanto em testes isolados quanto na operação integrada do sistema. A utilização da metodologia UVM contribuiu para aumentar a confiabilidade da validação, permitindo maior rastreabilidade e sistematização dos cenários testados.

8.1 Síntese

No que se refere à implementação física, o projeto foi submetido ao fluxo completo de síntese e implementação utilizando a ferramenta OpenLane, executada em ambiente containerizado via Docker. O fluxo adotado contemplou o processo RTL-to-GDSII, permitindo a geração do *layout* físico do circuito integrado utilizando o PDK *open source* SkyWater Sky130 (tecnologia CMOS de 130 nm).

O processo incluiu etapas de síntese lógica com Yosys e otimização via ABC, seguido pelas fases físicas conduzidas pelo OpenROAD, tais como *floorplanning*, *placement*, *Clock Tree Synthesis* (CTS), roteamento e Análise Estática de Temporização (STA). Posteriormente, foram utilizadas também as ferramentas Magice Netgen, garantindo conformidade com as regras de fabricação e consistência entre o *layout* físico e a *netlist* lógica.

A **Figura 18** apresenta a vista renderizada do *layout* físico do SoC após o processo completo de *place and route* no OpenLane. Observa-se a organização das macros principais - PicoRV32, AXI RAM e periféricos - conforme o *floorplan* definido, evidenciando a distribuição física dos blocos no *die*. Essa visualização confirma a correta integração estrutural do sistema e a materialização física da arquitetura proposta.

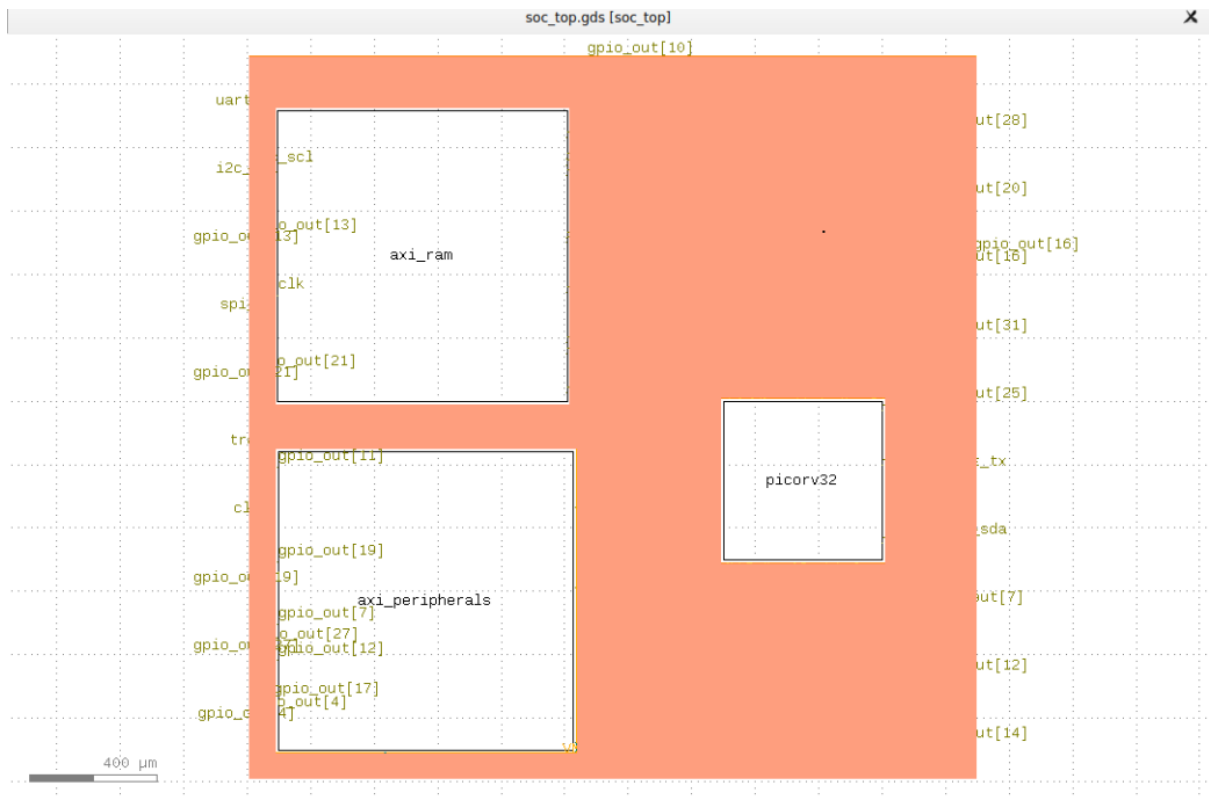


Figura 18 – Layout físico

A **Tabela 3** apresenta as principais métricas obtidas durante o fluxo de síntese como área ocupada, densidade de células, *slack* de temporização, contagem de portas lógicas e frequência máxima estimada. Esses resultados permitem avaliar o impacto de cada bloco na área total, no desempenho e na viabilidade física do sistema integrado.

design_name	total_runtime	DIEAREA_m²	CellPer_m²	wns	AND	DF	NAND	NO	OR	XOR	XNOR	MUX	Fmax MHz
picorv32	0h20m11s	0.25	44,7167	1.5	405	91	226	171	104	5	388	113	270
axi_peripherals	5h29m13s	0.872	53,7793	3.3	111	85	193	71	855	216	59	833	100.0
axi_ram	1h18m40s	0.827	53,8040	5.6	6	0	4	3	307	0	0	821	100.0
soc_top	0h33m27s	5.29	32.70	0.0	0	0	0	0	0	0	0	0	50.00

Tabela 3 – Métricas da síntese

Os resultados obtidos confirmam a viabilidade física da arquitetura proposta, demonstrando que o SoC pode ser implementado em tecnologia CMOS de 130nm dentro dos limites de área, desempenho e consumo estabelecidos.

9 Conclusão

Este trabalho apresentou o desenvolvimento e a verificação funcional de um SoC baseado no processador PicoRV32 para aplicações IoT, integrando processador, memórias, barramento AXI-Lite, periféricos e *bootloader* em uma arquitetura modular e escalável. A verificação foi conduzida por meio de testes individuais, integrados e utilizando a metodologia UVM, o que aumentou a confiabilidade da validação. Os resultados demonstraram que todos os módulos apresentaram funcionamento conforme os requisitos estabelecidos. A síntese confirmou a viabilidade da implementação, sem violações críticas de temporização.

A partir dos resultados obtidos, abre-se espaço para a realização de otimizações mais refinadas no projeto, com foco no aprimoramento individual de cada macrobloco do sistema. Estudos futuros podem explorar ajustes mais detalhados nos parâmetros de síntese e *place-and-route*, visando melhorar métricas como área, densidade, consumo de potência e desempenho temporal. Além disso, realizar a avaliação de diferentes estratégias de distribuição da rede de alimentação, analisando o impacto na integridade de sinal, queda de tensão e robustez do layout. Também podem ser investigadas variações na organização física dos blocos, buscando topologias que reduzam congestionamento e melhorem o roteamento global.

De modo geral, o projeto atingiu seus objetivos, evidenciando a aplicação prática de conceitos de arquitetura de SoC e verificação avançada para sistemas embarcados voltados à IoT.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] J. S. Yalli, M. H. Hasan and A. A. Badawi. **Internet of Things (IoT): Origins, Embedded Technologies, Smart Applications, and Its Growth in the Last Decade**. Disponível em: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10570411> Acesso em: 05 fev. 2026.
- [2] V. S. Chakravarthi and S. R. Koteswar. **System on Chip (SoC) Architecture: A Practical Approach**. Springer, Cham. https://doi.org/10.1007/978-3-031-36242-2_3.
- [3] ATZORI, Luigi; IERA, Antonio; MORABITO, Giacomo. The Internet of Things: A survey. *Computer Networks*, v. 54, n. 15, p. 2787–2805, 2010. DOI: 10.1016/j.comnet.2010.05.010
- [4] GUBBI, Jayavardhana et al. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, v. 29, n. 7, p. 1645–1660, 2013. DOI: 10.1016/j.future.2013.01.010.
- [5] WATERMAN, Andrew; ASANOVIĆ, Krste. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Berkeley: University of California, 2019. Disponível em: <https://riscv.org>. Acesso: 12 Fev 2026
- [6] WOLF, Clifford. *PicoRV32 – A Size-Optimized RISC-V CPU*. YosysHQ, 2016–2023. Disponível em: <https://github.com/YosysHQ/picorv32>
- [7] IEEE. *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2017, 2017. DOI: 10.1109/IEEESTD.2018.8299595.
- [8] PALNITKAR, Samir. *Verilog HDL: A Guide to Digital Design and Synthesis*. 2. ed. Upper Saddle River: Prentice Hall, 2003.
- [9] KEATING, Michael et al. *Reuse Methodology Manual for System-on-Chip Designs*. 3. ed. Boston: Springer, 2007. DOI: 10.1007/978-0-387-74050-6.
- [10] ARM. *AMBA AXI and ACE Protocol Specification*. ARM IHI 0022E, 2013. Disponível em: <https://developer.arm.com/documentation/102202/0300/AXI-protocol-overview?lang=en> . Acesso em 12 Fev 2026.